Creating custom Docker networks

In this train, we'll learn how to create and configure custom networks within Docker, tailoring them to specific project requirements.

Learning objectives

- Understand the process for creating and configuring custom networks within Docker.

Why Use Custom Docker Networks?

Docker provides a flexible networking model that allows containers to communicate with each other. By default, Docker creates a bridge network, but for more control, you can define custom networks tailored to your project's needs.

Creating a custom network allows you to:

- **Isolate containers** from the default bridge network.
- **Improve security** by restricting external access.
- **Enable better communication** between services in a multi-container application.
- **Customise network** settings such as subnets, gateways, and IP address ranges.

How to create a custom Docker network

Docker provides five network drivers: **Bridge** – Default, used for standalone containers. **Host** – Removes network isolation and allows the container to use the host's network. **Overlay** – Used for multi-host communication in Swarm mode. **None** – Completely disables networking and isolate a container from the host and other containers. **Ipvlan** - Provide full control over both IPv4 and IPv6 addressing. **Macvlan**Assign a MAC address to a container making it appear as a physical device on your network. For most use cases, bridge networks are used for inter-container communication.

1. Creating a Custom Bridge Network

- Docker's default networking mode is bridge mode which is often used when a container that needs to communicate with other containers on the same host. You

can create a custom bridge network using:

```
docker network create -d bridge my-bridge-network
```

> - `-d bridge` : Specifies that the network driver is a bridge.
> - `my-bridge-network` : Custom name for the network.

2. Connecting a Container to a Custom Network

- To start a container and connect it to the newly created network:

```
docker run -itd --network=my-bridge-network --name=my-container busybox
```

> - `--network=my-bridge-network` : Specifies the network to use.
> - `--name=my-container` : Assigns a custom name to the container.
> - `busybox` : The image to run.

3. Viewing and Managing Networks

- To list all available Docker networks, run:

```
docker network ls
```

- To inspect details about a specific network such as IP addresses, connected containers and network configurations:

```
docker network inspect my-bridge-network
```

4. Disconnecting and Removing Networks

- To disconnect a container from a network:

```
docker network disconnect my-bridge-network my-container
```

- To remove a custom network:

  ```
  docker network rm my-bridge-network
  ```

  > **Note:**
  >
  > You cannot remove a network if there are active containers connected to it.

Conclusion

Creating custom Docker networks allows you to manage container communication effectively. Understanding Docker networking is crucial for building scalable applications.

Key Takeaways

- Custom networks provide better isolation and communication control.
- Use `docker network create` to define new networks.
- Containers can be connected to custom networks using the `--network` flag.